

ARYABHATTA KNOWLEDGE UNIVERSITY, PATNA

BCA — 3rd SEMESTER EXAMINATION

OBJECT-ORIENTED PROGRAMMING USING C++

Paper Code: BCA-301 / 303301 | Full Marks: 60 | Time: 3 Hours

COMPLETE EXAM STUDY NOTES

Black & White | Point-by-Point | Easy English

PYQ Years Covered: **2014, 2017, 2018, 2019, 2020, 2021, 2022, 2024, 2025****CONTENTS**

1. Unit 1 — Introduction to OOP + Basics of C++
2. Unit 2 — Operator Overloading
3. Unit 3 — Inheritance
4. Unit 4 — Advanced Topics (Templates, Exception Handling, File Handling)
5. Most Repeated Questions (PYQ Frequency Table)
6. Important C++ Programs
7. Last Minute Cheat Sheet

MOST IMPORTANT VVI TOPICS (from PYQ Analysis)

- ★ Constructor & Destructor — 2014, 2017, 2018, 2019, 2020, 2021, 2024, 2025
- ★ Inheritance Types — 2014, 2017, 2018, 2019, 2021, 2022, 2024, 2025
- ★ Operator Overloading — 2014, 2017, 2018, 2019, 2021, 2022, 2025
- ★ Features of OOP / OOP vs POP — 2017, 2018, 2020, 2021, 2022, 2024
- ★ Friend Function — 2014, 2017, 2020, 2021, 2022, 2024, 2025
- ★ Virtual Functions & Dynamic Binding — 2017, 2019, 2020, 2021, 2022
- ★ Exception Handling — 2014, 2017, 2018, 2021, 2022, 2025
- ★ Templates — 2014, 2017, 2019, 2025 | File Handling — 2017, 2021, 2022

UNIT 1 — Introduction to Programming Paradigms & Basics of C++**SECTION A — MCQ (2 Marks Each)****Q1. Which is a feature of OOP?**

- (A) **Encapsulation ✓**
- (B) Goto statement
- (C) Global data
- (D) Recursion only

Ans: OOP features = Encapsulation, Inheritance, Polymorphism, Abstraction

★ PYQ: 2017, 2018, 2020

Q2. Default access specifier for class members in C++ is:

- (A) public
- (B) protected
- (C) **private ✓**
- (D) friend

Ans: class members are private by default in C++.

★ PYQ: 2019, 2022

Q3. Scope resolution operator (::) is used to:

- (A) Access private member
- (B) **Define member function outside class ✓**
- (C) Overload operators
- (D) Declare friend function

Ans: :: defines member function outside class and accesses global variables.

★ PYQ: 2019

Q4. cout stands for:

- (A) Class output
- (B) Character output
- (C) **Console output ✓**
- (D) Call output

Ans: cout = Console output. Used for printing to screen.

★ PYQ: 2018

Q5. Friend function is:

- (A) A member of class
- (B) **Not a member but has private access ✓**
- (C) A virtual function
- (D) A static function

Ans: Friend function is NOT a member but has access to private data.

★ PYQ: 2014

Q6. Hiding complexity is called:

- (A) **Abstraction ✓**
- (B) Encapsulation
- (C) Data hiding
- (D) Composition

Ans: Abstraction hides complexity and shows only essential features.

★ PYQ: 2018

Q7. Inline function is expanded:

- (A) At declaration
- (B) **At function call site ✓**

- (C) At end of program
- (D) At class definition

Ans: *Inline function body is pasted at call site — saves function call overhead.*

★ PYQ: 2017, 2020

Q8. Abstract class in C++ is used as:

- (A) Data storage
- (B) **Interface/Base class ✓**
- (C) File handling
- (D) Operator overloading

Ans: *Abstract class provides interface — has pure virtual functions.*

★ PYQ: 2019

SECTION B — 4-Mark Short Answers

★★ VVI

Q. What is data abstraction? Give example.

Definition:

1. Abstraction means hiding internal details and showing only what is necessary to the user.
2. It is one of the main features of OOP.

Example (Real life):

3. A car driver uses steering/brake without knowing how the engine works inside.

Example (C++):

```
class BankAccount { private: int balance; // hidden from outside public: void deposit(int amt) { balance += amt; } int getBalance() { return balance; } // only this is shown };
```

4. Here, 'balance' is abstracted (hidden). User can only use deposit() and getBalance().

Benefit: Simplifies interface. Reduces complexity.

★ PYQ: 2018, 2020, 2022

★★ VVI

Q. What is scope resolution operator? Write examples.

Definition:

1. The :: (double colon) operator is called scope resolution operator.

Uses:

2. Use 1: Define member function outside the class.

```
void MyClass::show() { cout << x; }
```

3. Use 2: Access global variable when local variable has same name.

```
int x = 5; // global void f() { int x = 10; cout << ::x; } // prints 5 (global)
```

4. Use 3: Access static member of class.

```
cout << ClassName::staticVar;
```

★ PYQ: 2017

★★ VVI

Q. What is friend function? Why is it needed?

Definition:

1. A friend function is NOT a member of a class, but it has access to private and protected members.
2. It is declared inside the class using keyword 'friend'.

Syntax (inside class):

```
class A { private: int x; friend void show(A obj); // declaration };
```

Definition (outside class, no 'friend' keyword):

```
void show(A obj) { cout << obj.x; } // accesses private x
```

Why needed?

3. To allow two classes to share private data.
4. Useful for operator overloading with two different class objects.

Important properties:

5. NOT inherited. 6. Has no 'this' pointer. 7. Can access private members directly.

★ PYQ: 2014, 2017, 2020, 2021, 2022, 2024, 2025

★★ VVI

Q. What are constructors? Explain types with examples.

Definition:

1. Constructor is a special function that is called AUTOMATICALLY when an object is created.
2. Same name as class. No return type.

Types:

3. Default Constructor — No arguments.

```
MyClass() { x = 0; }
```

4. Parameterized Constructor — Takes arguments.

```
MyClass(int a) { x = a; }
```

5. Copy Constructor — Creates object from another object.

```
MyClass(MyClass &obj) { x = obj.x; }
```

Destructor:

6. Destructor = ~ClassName(); Called when object is destroyed. Used for cleanup.

```
~MyClass() { cout << "Destroyed"; }
```

★ PYQ: 2014, 2017, 2018, 2019, 2020, 2021, 2024, 2025

Q. Difference between class and structure in C++.

1. In C++ struct, all members are PUBLIC by default.
2. In C++ class, all members are PRIVATE by default.
3. Class supports all OOP features. Struct is more like C struct.
4. Class is used for real OOP programming. Struct is used for simple data grouping.

Quick Table:

Feature	struct	class	Default access	public	private	Inheritance	Yes	Yes	Member functions	Yes
Usage	Simple data	Full OOP								

★ PYQ: 2018

SECTION C — 12-Mark Long Answers

★★ VVI

Q. What is OOP? Explain its features/characteristics.

Introduction:

1. OOP = Object-Oriented Programming. It is a programming style based on objects.
2. An object combines data (variables) and behavior (functions) together.

Feature 1 — Encapsulation:

3. Wrapping data and functions together inside a class.
4. Example: `class Student { private: int marks; public: void setMarks(int m){marks=m; } };`

Feature 2 — Abstraction:

5. Hiding complex implementation. Showing only what is necessary.
6. Example: User uses ATM without knowing internal banking logic.

Feature 3 — Inheritance:

7. Derived class gets properties of base class. Promotes code reuse.
8. Example: `class Dog : public Animal { ... };`

Feature 4 — Polymorphism:

9. Same function behaves differently in different situations.
10. Two types: Compile-time (overloading) and Runtime (virtual functions).

Feature 5 — Dynamic Binding:

11. Function call is linked to code at RUNTIME using virtual functions.

Feature 6 — Message Passing:

12. Objects communicate by calling each other's member functions.

Conclusion:

13. OOP makes programs modular, reusable, and easy to maintain.

★ PYQ: 2017, 2018, 2020, 2021, 2022, 2024

★★ VVI

Q. Write a C++ program showing all types of constructors and destructor.

Introduction:

1. Constructor: Same name as class. Auto-called when object is created. No return type.
2. Destructor: ~ClassName(). Auto-called when object is destroyed. For cleanup.

Complete C++ Program:

```
#include<iostream> using namespace std; class Student { int id; string name; public: // 1. Default
Constructor Student() { id=0; name="Unknown"; cout << "Default constructor called\n"; } // 2. Parameterized
Constructor Student(int i, string n) { id=i; name=n; cout << "Param constructor: " << name << "\n"; } // 3.
Copy Constructor Student(Student &s) { id=s.id; name=s.name; cout << "Copy constructor: " << name << "\n";
} void show() { cout << id << " " << name << "\n"; } // Destructor ~Student() { cout << "Destructor: " <<
name << "\n"; } }; int main() { Student s1; // Default Student s2(1, "Raj"); // Parameterized Student
s3(s2); // Copy s2.show(); s3.show(); return 0; }
```

Output:

Default constructor called Param constructor: Raj Copy constructor: Raj 1 Raj 1 Raj Destructor: Raj (in reverse order)...

Key Points:

3. Constructor order: Parent first, child last.
4. Destructor order: REVERSE of constructor (child first, parent last).

★ PYQ: 2014, 2017, 2018, 2019, 2020, 2021, 2024

UNIT 2 — Operator Overloading

SECTION A — MCQ (2 Marks Each)

Q1. Which operator CANNOT be overloaded in C++?

- (A) **. (dot) ✓**
- (B) + operator
- (C) [] operator
- (D) << operator

Ans: Operators that CANNOT be overloaded: . :: sizeof ?: .*

★ PYQ: Not verified

Q2. Binary operator overloaded as member function needs how many arguments?

- (A) 0
- (B) **1 ✓**
- (C) 2
- (D) 3

Ans: Member function: 1 argument (left operand = this, right = argument).

★ PYQ: 2017

Q3. Overloading << and >> must use:

- (A) Member function
- (B) **Friend function ✓**
- (C) Virtual function
- (D) Static function

Ans: << and >> must be friend functions — left operand is cout/cin (not the class).

★ PYQ: 2014, 2018

Q4. Which is valid syntax for operator overloading?

- (A) overload+(Complex c)
- (B) **operator+(Complex c) ✓**
- (C) +(Complex c)
- (D) define+(Complex c)

Ans: Syntax: return_type operator symbol (args) { ... }

★ PYQ: Not verified

Q5. Type conversion from class to basic type uses:

- (A) Constructor
- (B) **Conversion operator ✓**
- (C) Friend function
- (D) Destructor

Ans: Conversion operator: operator int() converts class object to int type.

★ PYQ: 2014

Q6. Function overloading is an example of:

- (A) Runtime polymorphism
- (B) **Compile-time polymorphism ✓**
- (C) Inheritance
- (D) Encapsulation

Ans: Function overloading = compile-time (static) polymorphism.

★ PYQ: 2020, 2021

SECTION B — 4-Mark Short Answers

★★ VVI

Q. What is operator overloading? Write its restrictions.**Definition:**

1. Operator overloading = giving new meaning to existing C++ operators for user-defined types (classes).
2. It allows objects to be used with +, -, *, etc., just like basic data types.

Syntax:

```
return_type operator symbol (arguments) { body }
```

Example:

```
Complex operator+(Complex c) { return Complex(real+c.real, imag+c.imag); }
```

Restrictions (cannot do):

3. Cannot overload: . :: sizeof ?: .* (these 5 operators NEVER)
4. Cannot change operator precedence or associativity.
5. Cannot create a NEW operator (e.g., ** for power).
6. At least one operand must be a user-defined (class) type.

★ PYQ: 2014, 2017, 2018, 2019, 2021, 2022, 2025

★★ VVI

Q. Overload << and >> operators for a class. Write syntax.**Why friend function?**

1. Left operand of << is 'cout' (ostream), not our class. So it cannot be a member function.
2. We must use friend function for << and >>.

Syntax for << (insertion operator):

```
friend ostream& operator<<(ostream& out, ClassName& obj) { out << obj.data; return out; }
```

Syntax for >> (extraction operator):

```
friend istream& operator>>(istream& in, ClassName& obj) { in >> obj.data; return in; }
```

Usage after overloading:

```
cout << obj1; // works! cin >> obj1; // works!
```

★ PYQ: 2014, 2018

★★ VVI

Q. What is function overloading? Write an example.**Definition:**

1. Function overloading = multiple functions with SAME name but DIFFERENT parameters.
2. Functions differ in: number of params, type of params, or order of params.

Example:

```
int add(int a, int b) { return a+b; } // 2 int float add(float a, float b) { return a+b; } // 2 float int
add(int a, int b, int c){ return a+b+c; } // 3 int // Calling: cout << add(3, 4); // calls 1st cout <<
add(3.5f, 2.5f); // calls 2nd cout << add(1, 2, 3); // calls 3rd
```

Benefit: No need for different function names. Improves readability.

★ PYQ: 2019, 2020, 2021, 2022

SECTION C — 12-Mark Long Answers

★★ VVI

Q. What is operator overloading? Write C++ program for overloading + operator for complex numbers.**Introduction:**

1. Operator overloading gives extra meaning to C++ operators for user-defined types.
2. We can add two complex numbers using + just like integers.

Syntax:

```
return_type operator+(arguments) { body }
```

Complete Program:

```
#include<iostream> using namespace std; class Complex { float real, imag; public: Complex(float r=0, float
i=0) { real=r; imag=i; } // Overload + operator (member function) Complex operator+(Complex c) { return
Complex(real + c.real, imag + c.imag); } // Overload << for easy output (friend function) friend ostream&
operator<<(ostream& out, Complex c) { out << c.real << "+" << c.imag << "i"; return out; } }; int main() {
Complex c1(3.0, 4.0), c2(1.5, 2.5); Complex c3 = c1 + c2; // calls operator+ cout << "c1 = " << c1 << endl;
// 3+4i cout << "c2 = " << c2 << endl; // 1.5+2.5i cout << "Sum = " << c3 << endl; // 4.5+6.5i return 0; }
```

Restrictions (important for exam):

3. Cannot overload: . :: sizeof ?: . *
4. Cannot change precedence/associativity.
5. At least one operand must be class type.

Conclusion:

6. Operator overloading makes user-defined types work naturally with operators.

★ PYQ: 2014, 2017, 2018, 2019, 2021, 2022, 2025

UNIT 3 — Inheritance

SECTION A — MCQ (2 Marks Each)

Q1. Inheritance allows derived class to:

- (A) Hide all data
- (B) **Acquire properties of base class ✓**
- (C) Delete base class
- (D) Create new operators

Ans: Inheritance: derived class gets (acquires) properties of base class.

★ PYQ: 2018

Q2. Constructors and destructors are NOT inherited because:

- (A) They are private
- (B) **They are object-specific initialization/cleanup ✓**
- (C) They are static
- (D) They are inline

Ans: Constructors/destructors are specific to the object of that class.

★ PYQ: 2014

Q3. Default inheritance type if not specified is:

- (A) public
- (B) protected
- (C) **private ✓**
- (D) friend

Ans: Default inheritance is private in C++.

★ PYQ: Not verified

Q4. Pure virtual function is declared as:

- (A) virtual void f() {}
- (B) void f() = 0;
- (C) **virtual void f() = 0; ✓**
- (D) abstract void f();

Ans: Pure virtual: virtual void f() = 0; — makes class abstract.

★ PYQ: 2017, 2021

Q5. Diamond problem occurs in:

- (A) Single inheritance
- (B) Multilevel inheritance
- (C) **Multiple inheritance ✓**
- (D) Hierarchical inheritance

Ans: Diamond: class D inherits from B and C, both of which inherit from A.

★ PYQ: 2020, 2022

Q6. Which specifier makes base class members accessible in derived class but not outside?

- (A) public
- (B) private
- (C) **protected ✓**
- (D) friend

Ans: protected: accessible in derived class but not outside the class.

★ PYQ: 2018

SECTION B — 4-Mark Short Answers

★★ VVI

Q. What is inheritance? Write its types.**Definition:**

1. Inheritance = a derived class acquires properties and behaviors of a base class.
2. Promotes CODE REUSE — no need to rewrite existing code.

Syntax:

```
class DerivedClass : access_specifier BaseClass { ... };
```

Types:

3. Single: One base, one derived. `class B : public A { };`
4. Multiple: Multiple bases, one derived. `class C : public A, public B { };`
5. Multilevel: Chain A->B->C. `class B:public A{ };` `class C:public B{ };`
6. Hierarchical: One base, multiple derived. `class B:public A{ };` `class C:public A{ };`
7. Hybrid: Combination of two or more types. May cause diamond problem.

★ PYQ: 2014, 2017, 2018, 2019, 2021, 2022, 2024, 2025

★★ VVI

Q. What is virtual function? Why is it used?**Definition:**

1. A virtual function is declared in base class with keyword 'virtual'.
2. It is overridden in derived class.
3. Correct version is called at RUNTIME based on object type = Runtime Polymorphism.

Syntax:

```
// Base class: virtual void show() { cout << "Base"; } // Derived class: void show() { cout << "Derived"; }
```

Usage:

```
Base *ptr = new Derived(); ptr->show(); // Output: Derived (runtime decision)
```

Without virtual:

4. Without virtual: Base class function is always called (static binding).
5. With virtual: Correct derived class function is called (dynamic binding).

★ PYQ: 2017, 2019, 2020, 2021, 2022

★★ VVI

Q. What is abstract class? What are its properties?**Definition:**

1. An abstract class has at least ONE pure virtual function.
2. A pure virtual function is declared as: `virtual void f() = 0;`
3. Abstract class CANNOT be instantiated (cannot create object directly).

Properties:

4. It provides a common base for related classes.
5. Derived class MUST implement all pure virtual functions (otherwise derived class is also abstract).
6. Can have constructors, data members, and normal functions too.

Example:

```
class Shape { // abstract class public: virtual void draw() = 0; // pure virtual }; class Circle : public Shape { public: void draw() { cout << "Drawing Circle"; } // must implement }; // Shape s; ERROR! Cannot create object of abstract class Circle c; // OK
```

★ PYQ: 2017, 2018, 2019, 2022, 2024

★★ VVI

Q. Explain access specifiers with inheritance table.**Three access specifiers: public, protected, private****When Base is inherited as PUBLIC:**

1. public members → remain public in derived
2. protected members → remain protected in derived
3. private members → NOT directly accessible in derived

When Base is inherited as PROTECTED:

4. public members → become protected in derived
5. protected members → remain protected in derived

When Base is inherited as PRIVATE (default):

6. public members → become private in derived
7. protected members → become private in derived

KEY RULE: private members of base class are NEVER directly accessible in any derived class.

★ PYQ: 2018

SECTION C — 12-Mark Long Answers

★★ VVI

Q. Explain all types of inheritance with C++ programs.

Introduction: Inheritance allows code reuse by letting one class get properties of another.

1. Single Inheritance:

```
class Animal { public: void eat(){cout<<"Eating";} }; class Dog : public Animal { public: void bark(){cout<<"Barking";} }; int main(){Dog d; d.eat(); d.bark();}
```

2. Multiple Inheritance:

```
class A {public: void f1(){cout<<"A";}}; class B {public: void f2(){cout<<"B";}}; class C : public A, public B {}; int main(){C c; c.f1(); c.f2();}
```

3. Multilevel Inheritance:

```
class A {public: void f1(){cout<<"A";}}; class B : public A {public: void f2(){cout<<"B";}}; class C : public B {public: void f3(){cout<<"C";}}; int main(){C c; c.f1(); c.f2(); c.f3();}
```

4. Hierarchical Inheritance:

```
class Vehicle {public: void run(){cout<<"Running";}}; class Car : public Vehicle {}; class Bike : public Vehicle {}; int main(){Car c; Bike b; c.run(); b.run();}
```

5. Hybrid Inheritance:

1. Combination of two or more inheritance types.
2. Can cause diamond problem (solved by virtual base class).

Conclusion: C++ supports all 5 inheritance types for maximum code reuse.

★ PYQ: 2014, 2017, 2018, 2019, 2021, 2022, 2024, 2025

★★ VVI

Q. What is runtime polymorphism? Demonstrate with virtual function program.

Introduction:

1. Polymorphism = same function behaves differently for different objects.
2. Runtime polymorphism: correct function is chosen at RUNTIME using virtual functions.

How it works — Step by step:

3. Step 1: Declare function as 'virtual' in base class.
4. Step 2: Override function in derived class.
5. Step 3: Create base class pointer pointing to derived object.
6. Step 4: Call function through pointer — correct version is called at runtime.

C++ Program:

```
#include<iostream> using namespace std; class Shape { public: virtual void draw() { cout << "Drawing Shape" << endl; } virtual ~Shape() {} // virtual destructor }; class Circle : public Shape { public: void draw() { cout << "Drawing Circle" << endl; } }; class Rectangle : public Shape { public: void draw() { cout << "Drawing Rectangle" << endl; } }; int main() { Shape *ptr; Circle c; Rectangle r; ptr = &c; ptr->draw(); // Output: Drawing Circle (runtime) ptr = &r; ptr->draw(); // Output: Drawing Rectangle (runtime) return 0; }
```

Key difference:

7. WITHOUT virtual: Base class function is always called (compile-time / static binding).
8. WITH virtual: Correct derived class function is called (runtime / dynamic binding).

Conclusion: Virtual functions are the heart of runtime polymorphism in C++.

★ PYQ: 2017, 2019, 2020, 2021, 2022

UNIT 4 — Advanced Topics (Virtual Base Class, Templates, Exception & File Handling)

SECTION A — MCQ (2 Marks Each)

Q1. Virtual base class is used to solve:

- (A) Single inheritance
- (B) **Diamond problem ✓**
- (C) File handling issue
- (D) Template error

Ans: Virtual base class prevents duplicate copies of base in diamond inheritance.

★ PYQ: 2020, 2022

Q2. Exception handling keywords in C++ are:

- (A) **try, catch, throw ✓**
- (B) begin, error, end
- (C) test, handle, raise
- (D) check, trap, signal

Ans: Exception handling: try{} throw catch() — these three keywords.

★ PYQ: 2017

Q3. For reading from a file in C++, use:

- (A) ofstream
- (B) **ifstream ✓**
- (C) fstream
- (D) iostream

Ans: ifstream = input file stream (for reading files from disk).

★ PYQ: 2017, 2022

Q4. What is a template in C++?

- (A) Overloaded function
- (B) **Generic code for any data type ✓**
- (C) Virtual function
- (D) Abstract class

Ans: Template = generic function/class that works with any data type.

★ PYQ: 2014, 2017, 2019, 2025

Q5. seekg() function is used to:

- (A) Write to file
- (B) **Move file read pointer to a position ✓**
- (C) Close file
- (D) Open file

Ans: seekg() moves the get (read) pointer to a specific position in file.

★ PYQ: 2022

Q6. 'this' pointer points to:

- (A) Base class object
- (B) Derived class
- (C) **Current object inside member function ✓**
- (D) Static member

Ans: 'this' is an implicit pointer to the current object in member functions.

★ PYQ: 2022

SECTION B — 4-Mark Short Answers

★★ VVI

Q. What is virtual base class? How does it solve diamond problem?**Diamond Problem:**

1. Class B and C both inherit from A.
2. Class D inherits from both B and C.
3. Result: Two copies of A exist in D → AMBIGUITY (diamond problem).

```
A / \ B C \ / D <- D has two copies of A! ERROR
```

Solution — Virtual Base Class:

4. Use 'virtual' keyword in B and C when inheriting from A.

```
class B : virtual public A { }; class C : virtual public A { }; class D : public B, public C { }; // Now D
has only ONE copy of A. Problem solved!
```

Result: Only ONE shared copy of A exists in D. No ambiguity.

★ PYQ: 2020, 2022, 2025

★★ VVI

Q. What is exception handling? Explain try, catch, throw.**Definition:**

1. Exception handling = mechanism to handle runtime errors without crashing the program.

Keywords:

2. try block: Contains code that might cause an error.
3. throw: Used to raise (signal) an exception.
4. catch block: Catches and handles the exception.

Syntax:

```
try { // code that might fail throw value; // throw exception } catch(data_type variable) { // handle the
error }
```

Example:

```
try { if(b == 0) throw 0; // throw integer cout << a/b; } catch(int e) { cout << "Error: Division by
zero!"; }
```

5. catch(...) catches ALL types of exceptions.
6. Multiple catch blocks can handle different exception types.

★ PYQ: 2014, 2017, 2018, 2021, 2022, 2025

★★ VVI

Q. What are templates? Write syntax for function template and class template.**Definition:**

1. Template = a blueprint for creating generic functions or classes.
2. One template works with int, float, char, string — any data type.

Function Template Syntax:

```
template<typename T> T functionName(T a, T b) { return a + b; }
```

Class Template Syntax:

```
template<class T> class ClassName { T data; public: void setData(T d) { data = d; } T getData() { return
data; } };
```

Benefits:

3. Write once, use for any data type.
4. Reduces code duplication.
5. Type-safe generic programming.

★ PYQ: 2014, 2017, 2019, 2025

★★ VVI

Q. Explain file handling in C++. What are different file modes?**Definition:**

1. File handling = storing and reading data from disk files permanently.

File Stream Classes:

2. ifstream — for READING from file (input).
3. ofstream — for WRITING to file (output).

4. fstream — for BOTH reading and writing.

File Modes:

ios::in — Read mode ios::out — Write mode ios::app — Append (add at end) ios::trunc — Truncate (clear file)
ios::binary — Binary mode ios::ate — Open and move to end

Important functions for random access:

5. seekg(pos) — move read pointer to position 'pos'.
6. seekp(pos) — move write pointer.
7. tellg() — returns current read pointer position.
8. tellp() — returns current write pointer position.

★ PYQ: 2017, 2021, 2022

SECTION C — 12-Mark Long Answers

★★ VVI

Q. What is exception handling? Write C++ program with user-defined exception class.

Introduction:

1. Exception handling handles runtime errors gracefully. Program does not crash.
2. Normal code and error-handling code are SEPARATED.

Three keywords:

3. try: Contains risky code. throw: Signals/raises exception. catch: Handles it.

C++ Program with user-defined exception:

```
#include<iostream> using namespace std; // User-defined exception class
class DivisionError { string message; public: DivisionError(string m) { message = m; } void display() { cout << "ERROR: " << message << endl; } };
float divide(float a, float b) { if(b == 0) throw DivisionError("Cannot divide by zero!"); // throw object
return a / b; } int main() { float x = 10, y = 0; try { float result = divide(x, y); cout << "Result: " << result << endl; } catch(DivisionError& e) { // catch object e.display(); } catch(...) { cout << "Some unknown error occurred" << endl; } cout << "Program continues normally..." << endl; return 0; }
```

Output:

ERROR: Cannot divide by zero! Program continues normally...

Key Points for exam:

4. throw can throw any value: integer, string, or class object.
5. catch(...) is called 'catch-all' handler.
6. Program continues after catch block — does NOT stop.

★ PYQ: 2014, 2017, 2018, 2021, 2022, 2025

★★ VVI

Q. What are templates? Write programs for function template and class template.

Introduction:

1. Template = generic programming. One code, multiple data types.
2. Two types: Function template and Class template.

Program 1 — Function Template:

```
#include<iostream> using namespace std; template<typename T> T findMax(T a, T b) { return (a > b) ? a : b; }
int main() { cout << findMax(10, 20) << endl; // int: 20 cout << findMax(3.5, 2.1) << endl; // float: 3.5 cout << findMax('A', 'Z') << endl; // char: Z return 0; }
```

Program 2 — Class Template:

```
#include<iostream> using namespace std; template<class T> class Pair { T first, second; public: Pair(T a, T b) { first=a; second=b; } T getMax() { return (first > second) ? first : second; } };
int main() { Pair<int> p1(3, 7); cout << "Max int: " << p1.getMax() << endl; // 7 Pair<float> p2(2.5, 1.8); cout << "Max float: " << p2.getMax() << endl; // 2.5 return 0; }
```

Advantages:

3. Write once, use for any data type.
4. No code duplication.
5. Type-safe at compile time.

Conclusion: Templates are the foundation of generic programming in C++.

★ PYQ: 2014, 2017, 2019, 2025

★★ VVI

Q. Write a complete C++ program demonstrating file handling (write, read, append).**Introduction:**

1. File handling lets programs save data permanently to disk.

Complete Program:

```
#include<iostream> #include<fstream> using namespace std; int main() { // ---- STEP 1: WRITE to file ----
ofstream fout("student.txt"); fout << "Name: Raj" << endl; fout << "Roll: 101" << endl; fout << "Marks: 85"
<< endl; fout.close(); cout << "Data written successfully." << endl; // ---- STEP 2: READ from file ----
ifstream fin("student.txt"); string line; cout << "--- Reading file ---" << endl; while(getline(fin, line))
{ cout << line << endl; } fin.close(); // ---- STEP 3: APPEND to file ---- fstream fs("student.txt",
ios::app); fs << "City: Patna" << endl; fs.close(); cout << "Data appended." << endl; return 0; }
```

Random access functions:

2. seekg(n, ios::beg): move read pointer n bytes from beginning.
3. seekg(0, ios::end): move to end of file.
4. tellg(): returns current read pointer position.

Key note:

5. Always call .close() after file operations to save data properly.

★ PYQ: 2017, 2021, 2022

MOST REPEATED QUESTIONS — PYQ FREQUENCY TABLE

Topic	Type	Years (verified from uploaded PYQs)	Count
Constructor & Destructor	12M/4M	2014,2017,2018,2019,2020,2021,2024,2025	★★★ 8+
Inheritance Types + Program	12M	2014,2017,2018,2019,2021,2022,2024,2025	★★★ 8
Operator Overloading (+)	12M/4M	2014,2017,2018,2019,2021,2022,2025	★★★ 7
Friend Function + Program	12M/4M	2014,2017,2020,2021,2022,2024,2025	★★★ 7
Features of OOP / OOP vs POP	12M/4M	2017,2018,2020,2021,2022,2024	★★ 6
Exception Handling	12M/4M	2014,2017,2018,2021,2022,2025	★★ 6
Virtual Functions + Polymorphism	12M/4M	2017,2019,2020,2021,2022	★★ 5
Abstract Class	4M/12M	2017,2018,2019,2022,2024	★★ 5
Templates (Function + Class)	12M/4M	2014,2017,2019,2025	★★ 4
Copy Constructor	4M/12M	2014,2019,2021,2025	★ 4
File Handling	12M	2017,2021,2022	★ 3
Virtual Base Class (Diamond)	4M/12M	2020,2022,2025	★ 3
this pointer	2M/4M	2022	1
Scope Resolution (::)	2M/4M	2017,2019	2

★★★ = Extremely VVI | ★★ = Very Important | ★ = Important | All PYQ years verified from uploaded papers only.

IMPORTANT C++ PROGRAMS

1. Friend Function Program

★ PYQ: 2024, 2025

```
#include<iostream> using namespace std; class Box { int length; public: Box(int l) { length = l; } friend void printLength(Box b); // friend declaration }; void printLength(Box b) { cout << "Length = " << b.length; // accesses private member } int main() { Box b(10); printLength(b); } // Output: Length = 10
```

2. Copy Constructor Program

★ PYQ: 2019, 2021

```
#include<iostream> using namespace std; class Student { int id; string name; public: Student(int i, string n) { id=i; name=n; } Student(Student &s) { id=s.id; name=s.name; // copy constructor cout << "Copy created: " << name << endl; } void show() { cout << id << " " << name << endl; } }; int main() { Student s1(1, "Raj"); Student s2(s1); // copy constructor called s2.show(); }
```

3. Single Inheritance Program

★ PYQ: 2024

```
#include<iostream> using namespace std; class Person { public: string name; int age; void setData(string n, int a){ name=n; age=a; } void show(){ cout<<"Name: "<<name<<" Age: "<<age<<endl; } }; class Student : public Person { public: int roll; void setRoll(int r){ roll=r; } void display(){ show(); cout<<"Roll: "<<roll<<endl; } }; int main(){ Student s; s.setData("Raj",20); s.setRoll(101); s.display(); }
```

4. Function Template — Find Product

★ PYQ: 2025

```
#include<iostream> using namespace std; template<typename T> T product(T a, T b) { return a * b; } int main() { cout << product(4, 5) << endl; // int: 20 cout << product(2.5, 3.0) << endl; // float: 7.5 return 0; }
```

5. Virtual Function — Runtime Polymorphism

★ PYQ: 2017, 2019, 2022

```
#include<iostream> using namespace std; class Animal { public: virtual void sound() { cout << "Animal sound" << endl; } }; class Dog : public Animal { public: void sound() { cout << "Dog: Woof!" << endl; } }; class Cat : public Animal { public: void sound() { cout << "Cat: Meow!" << endl; } }; int main() { Animal *ptr; Dog d; Cat c; ptr = &d; ptr->sound(); // Dog: Woof! ptr = &c; ptr->sound(); // Cat: Meow! }
```

6. Exception Handling with Multiple Catch

★ PYQ: 2014, 2021, 2025

```
#include<iostream> using namespace std; int main() { int choice; cout << "Enter 1 (int error), 2 (float error), 3 (string error): "; cin >> choice; try { if(choice==1) throw 10; // int exception if(choice==2) throw 3.14; // float exception if(choice==3) throw "Error!"; // string exception } catch(int e) { cout << "Int exception: " << e; } catch(float e) { cout << "Float exception: " << e; } catch(const char* e) { cout << "String: " << e; } catch(...) { cout << "Unknown exception"; } }
```

LAST MINUTE CHEAT SHEET

Box 1 — Key One-Liner Definitions

Term	One-Liner Definition
Class	Blueprint for creating objects. Defines data + functions.
Object	Instance of a class. Actual entity in memory.
Encapsulation	Wrapping data + functions together. Hides internal details.
Abstraction	Hiding complexity. Showing only what is necessary.
Inheritance	Derived class gets properties of base class. Code reuse.
Polymorphism	Same function behaves differently in different contexts.
Constructor	Auto-called when object is created. No return type. Same name as class.
Destructor	~ClassName(). Auto-called when object is destroyed. For cleanup.
Virtual Function	Declared with 'virtual'. Enables runtime polymorphism.
Pure Virtual	virtual void f() = 0; Makes class abstract. MUST be overridden.
Abstract Class	Has at least 1 pure virtual function. Cannot be instantiated.
Friend Function	NOT a member. Has access to private members of class.
this pointer	Implicit pointer to current object inside member functions.
Template	Generic function/class that works with any data type.
try/catch/throw	Error handling: try=risky code, throw=signal, catch=handle.
ifstream	Input file stream — for reading from files.
ofstream	Output file stream — for writing to files.
Virtual base class	Prevents duplicate base class copies in multiple inheritance.
Static member	Shared by ALL objects of class. Belongs to class, not object.
Inline function	Body pasted at call site. Faster. Use for small functions.

Box 2 — C vs C++ Differences (PYQ 2020, 2025)

Feature	C Language	C++ Language
Programming style	Procedural (POP)	Object-Oriented (OOP)
Classes & Objects	Not supported	Fully supported
Function overloading	Not supported	Supported
Operator overloading	Not supported	Supported
Inheritance	Not supported	Supported
Exception handling	No (only errno)	Yes (try/catch/throw)
Input/Output	scanf() / printf()	cin / cout
Data security	Less (global data OK)	More (private/protected)
Memory management	malloc/free	new/delete

Box 3 — Syntax Quick Reference

Concept	Syntax
Class	<code>class Name { private: int x; public: void f(); };</code>
Constructor	<code>ClassName(int a) { x=a; }</code>
Destructor	<code>~ClassName() { /* cleanup */ }</code>

Inheritance	<code>class D : public B { };</code>
Virtual function	<code>virtual void show() { }</code>
Pure virtual	<code>virtual void show() = 0;</code>
Friend function	<code>friend void f(ClassName obj); // inside class</code>
Op overload	<code>RetType operator+(ClassName c) { return ...; }</code>
Fn template	<code>template<typename T> T add(T a, T b) { return a+b; }</code>
Class template	<code>template<class T> class Pair { T x; };</code>
Exception	<code>try { throw e; } catch(Type e) { handle; }</code>
File write	<code>ofstream f("file.txt"); f << "data"; f.close();</code>
File read	<code>ifstream f("file.txt"); getline(f, line); f.close();</code>
Virtual base	<code>class B : virtual public A { };</code>
this pointer	<code>this->x = a; or return *this;</code>

Box 4 — Common Exam Mistakes to AVOID

1. DON'T say base class pointer CANNOT point to derived class — it CAN (with virtual functions).
2. DON'T confuse constructor (no ~) with destructor (~ClassName).
3. DON'T say friend function IS a member — it is NOT a member.
4. DON'T forget: destructors are called in REVERSE order of constructors.
5. DON'T overload: . :: sizeof ?: .* — these 5 CANNOT be overloaded.
6. DON'T confuse ifstream (read) with ofstream (write).
7. DON'T say abstract class can be instantiated — it CANNOT.
8. DON'T say static member belongs to object — it belongs to CLASS.
9. DON'T forget: friend function has NO 'this' pointer.
10. DON'T say virtual needs runtime — it uses vtable mechanism internally.

Box 5 — Memory Tricks

- ♦ OOP features: E-A-I-P-D-M = Encapsulation, Abstraction, Inheritance, Polymorphism, Dynamic binding, Message passing
- ♦ Constructor order: PARENT first, CHILD last | Destructor: REVERSE (CHILD first, PARENT last)
- ♦ 5 operators you CANNOT overload: . :: sizeof ?: .*
- ♦ ifstream = IN from file (read) | ofstream = OUT to file (write) | fstream = BOTH
- ♦ Virtual = Runtime (late/dynamic binding) | Non-virtual = Compile time (early/static binding)
- ♦ Abstract class = has at least 1 pure virtual (=0) function. Cannot create object directly.
- ♦ Friend: declared INSIDE class with 'friend' keyword. Defined OUTSIDE. NOT a member.
- ♦ Template syntax: template<typename T> — T can be int, float, char, any type.
- ♦ Diamond problem: two paths from D to A. Solution: virtual base class.

END OF NOTES — BCA-301 OOP C++ | AKU Patna

All PYQ years are verified from uploaded exam papers only. Not guaranteed. Study all topics for complete preparation.